

Data Structure Lab2

| 叶锦灏 24300750085

| Summary

In this lab, I implemented a integer calculator, being able to treat with five operators and braces. Using the stack data structure, I implemented the `infix2postfix` algorithm as well as the `postfix2result` algorithm.

| Data Structure Implement

I firstly implemented a stack data structure class to support the algorithms next. Here's the core code for the stack:

```
template <typename T>
class Stack{
    private:
        size_t size;
        T* data;
        int top;
    public:
        Stack(size_t s): size(s), data(new T[size]), top(-1){}

        bool isFull() const {
            return top == static_cast<int>(size) - 1;
        }
}
```

```

    bool isEmpty() const {
        return top == -1;
    }

    void push(T x){
        if(isFull())throw(std::overflow_error("stack overflow!"));
        data[++top] = x;
        return;
    }

    T pop(){
        if(isEmpty())throw(std::overflow_error("stack underflow!"));
        return data[top--];
    }

    T readTop(){
        return data[top];
    }

    int getCurrentSize(){
        return top + 1;
    }
};

```

Stack follows the rule of last in first out, with two core methods: `push` and `pop`. It matches our problem to transfer infix to postfix and to transfer postfix to the calculated result.

Module Design of the Calculator

Three core functions are included in the calculator program:

- `parse` : to parse the input expression into tokens for following procession.
- `infix2postfix` : to transfer our natural mathematical expression infix form into the postfix form, to match the features of stack data structure.
- `postfix2result` : to calculate out the final result of the postfix-formed expression.

The core design and their design thoughts are shown below.

The Parser

```
vector<string> parse(string expr){
    vector<string> result;
    string num;
    string word;

    for(char c : expr){
        // 如果是数字，连续处理
        if(isdigit(c)){
            num+=c;
            continue;
        }

        if(!isdigit(c) && num.size() != 0){
            result.push_back(num);
            num.clear();
        }

        if(isspace(c))continue;

        if(isalpha(c)){
            word += c;
            continue;
        }

        if(!isalpha(c) && word.size() != 0){
            result.push_back(word);
            word.clear();
        }

        result.push_back(string(1,c));
    }

    if(!num.empty())result.push_back(num);
    if(!word.empty()) result.push_back(word);
}
```

```
    return result;
}
```

The parser parse our expression like `3 * (5 + 6)` into a sequence of symbols: `{"3", "*", "(", "5", "+", "6", ")"}`. Three kinds of tokens need to be process seperately: numbers, like `215`; space; and the operator `mod`. The digits of a number require to be put together, rather than seperated. Spaces should be ignored. The operator `mod` should also not be seperated. Tokens besides them are all single-character, can be directly put into the result: `+-*/( )`.

Infix to Postfix

```
vector<string> infix2postfix(vector<string> expr_infix){
    Stack<string> operators = Stack<string>(100);
    vector<string> output;

    for(string token : expr_infix){
        switch (classify(token))
        {
            case 0: // 数字
                output.push_back(token);
                break;

            case 1: // 操作符
                while (!operators.isEmpty() && isOperator(operators.readTop()) &&
                    getOpPriority(operators.readTop()) ≥ getOpPriority(token)) {
                    output.push_back(operators.pop());
                }
                operators.push(token);
                break;

            case 2: // 左括号
                operators.push(token);
                break;

            case 3: // 右括号
                while(operators.readTop()≠"("){
```

```

        output.push_back(operators.pop());
    }
    operators.pop();
    break;
}
}

while(!operators.isEmpty()){
    output.push_back(operators.pop());
}

return output;
}

```

I adapted the stack structure restoring the operators to transfer infix form into postfix form. Following the rules of:

1. Operands are immediately placed onto the output and operators are placed onto the stack .
2. When the operator on the top of the stack that has higher priority than the input operator, it is popped.
3. If we see a right parenthesis, then we pop the stack, writing symbols until we encounter a left parenthesis.
4. If we read the end of input, we pop the stack until it is empty.

So in the `infix2postfix` function, we firstly classify the token by a classification function, then process them by the rules.

Postfix 2 Result

```

int postfix2result(vector<string> expr_postfix){
    Stack<int> numbers = Stack<int>(100);
    for(string token : expr_postfix){
        if(isNum(token)){
            numbers.push(std::stoi(token));

```

```

        continue;
    }

    if(token == "+"){
        numbers.push(numbers.pop() + numbers.pop());
        continue;
    }

    if(token == "-"){
        int b = numbers.pop();
        int a = numbers.pop();
        numbers.push(a-b);
        continue;
    }

    if(token == "*"){
        numbers.push(numbers.pop() * numbers.pop());
        continue;
    }

    if(token == "/"){
        int b = numbers.pop();
        int a = numbers.pop();
        if(b == 0) throw std::runtime_error("Division by zero!");
        numbers.push(a/b);
        continue;
    }

    if(token == "mod"){
        int b = numbers.pop();
        int a = numbers.pop();
        if(b == 0) throw std::runtime_error("Mod by zero!");
        numbers.push(a%b);
        continue;
    }
}

if(numbers.getCurrentSize()==1){
    return numbers.pop();
}else throw("invalid expression!");
}

```

I adapted the stack storing numbers to calculate the postfix form expression. The calculation rules is simple, we just push the numbers into the stack, and pop them when we see an operator. It should be noted that some potential errors should be processed, like division by zero and mod by zero. Finally, if the function can't process the expression well, it means that the input is invalid.

I also implemented several supporting functions to make the codes of core functions simpler.

Conclusion and Weakness

The calculator is in a simple and primitive form, which only support integer and limited operators. In the future, it's expected to support float number, multi level brackets and more operations.